# Container

## 什麼是容器(Container)？

你在自己的筆電上開發了一個服務，跑起來完全沒問題。部署到伺服器上，卻在某個 library 版本不對的地方爆掉。換個新同事加入，光是把開發環境搭起來就花了半天。

這是大多數工程師熟悉的痛苦，而容器是用來解決這個問題的技術。

容器把應用程式和它需要的所有東西——程式碼、執行環境、library、設定——打包成一個獨立的單元。這個單元在任何地方都能以完全相同的方式執行。你的筆電、測試環境、生產伺服器，行為都一致。

在系統設計的脈絡下，容器的意義不只是「讓部署更方便」。它是現代水平擴展架構的基礎——讓你能快速複製服務實例、隔離故障範圍，並且有效利用硬體資源。了解容器的運作方式，以及什麼時候該在設計中引入容器化，是系統設計面試中展現架構成熟度的關鍵。

## 虛擬機器 vs. 容器

在容器普及之前，隔離和部署的標準方案是虛擬機器（Virtual Machine，VM）。理解兩者的差異，你才能在面試中說清楚為什麼容器在大多數現代場景下是更好的選擇。

**虛擬機器**是在一台實體機器上模擬出整個電腦，包含獨立的作業系統核心（kernel）。每台 VM 都要跑一份完整的 OS，佔用幾 GB 的記憶體，啟動要花幾分鐘，而且虛擬化硬體的 overhead 相當可觀。好處是完全隔離——一台 VM 裡發生的事完全影響不到另一台。

**容器**走了一條不同的路。它不是模擬整台電腦，而是共用宿主機（host machine）的 OS 核心，只把應用程式和必要的 library 隔離開來。Linux 核心提供的 namespace 機制讓每個容器有自己的行程空間、網路介面和檔案系統視圖；cgroups 機制則限制每個容器可以使用的 CPU 和記憶體資源。

這個差異帶來的結果是：

| | 虛擬機器 | 容器 |
|---|---|---|
| 啟動時間 | 幾分鐘 | 幾秒（通常幾百毫秒） |
| 記憶體佔用 | 幾 GB（包含完整 OS） | 幾十到幾百 MB |

fullmoal.org

| | 虛擬機器 | 容器 |
|---|---|---|
| 隔離程度 | 完整的 kernel 隔離 | 行程層面的隔離，共用 kernel |
| 適合場景 | 強安全隔離、跑不同 OS | 微服務、快速擴展、高密度部署 |

在系統設計面試中，除非題目明確涉及強安全隔離（例如讓不同租戶的程式碼在同一台機器上執行），不然大多數時候你談的都是容器，而不是 VM。

## Docker：容器的基本單位

Docker 是目前最廣泛使用的容器技術。理解 Docker 的核心概念，能讓你在面試中準確地用術語溝通。

### Image 和 Container

**Image**（鏡像）是容器的藍圖，是唯讀的靜態檔案，定義了容器啟動後的狀態——包含作業系統的基礎層、應用程式的程式碼和所有相依套件。你可以把 image 想成類別（class）的定義。

**Container**（容器）是 image 的執行實例。就像一個類別可以建立多個物件，一個 image 可以同時跑多個容器。每個容器有自己的執行狀態和可寫的檔案系統層，但底層的 image 是共用的。

你用 **Dockerfile** 來定義 image 的建置方式：

```
# 從官方的 Node.js image 開始
FROM node:20-alpine

# 設定工作目錄
WORKDIR /app

# 先複製 package.json，讓 Docker 快取 npm install 的結果
COPY package*.json ./
RUN npm ci --only=production

# 再複製應用程式程式碼
COPY . .

# 暴露 port
EXPOSE 3000
```

buildmoat.org

```
# 容器啟動時執行的命令
CMD ["node", "server.js"]
```

```
# 建置 image
docker build -t my-service:1.0 .

# 以這個 image 啟動一個容器
docker run -p 3000:3000 my-service:1.0

# 同時啟動三個容器（水平擴展）
docker run -p 3001:3000 my-service:1.0
docker run -p 3002:3000 my-service:1.0
docker run -p 3003:3000 my-service:1.0
```

### Image 分層的概念

Image 是由多層（layer）堆疊而成的。Dockerfile 中的每個指令都產生一層，Docker 會快取每一層。如果你只改了應用程式程式碼，Docker 不需要重新安裝所有 npm 套件——它直接從快取拿取 `npm ci` 那一層，只重新建置有改動的部分。

這讓建置速度快得多，也讓儲存更有效率——多個 image 共用相同的基礎層時，磁碟上只存一份。

**Image Registry** 是存放和分發 image 的中央倉庫。Docker Hub 是公開的 registry；實際生產環境通常用 AWS ECR、Google Artifact Registry 或自架的 Harbor。CI/CD pipeline 把新建置的 image 推送到 registry，部署時再從 registry 拉取。

buildneat.org

## 為什麼容器對系統設計很重要

在回答面試題目時，你提到容器不只是在展示技術細節，而是在說明你的系統如何實現一些關鍵的架構屬性。

### 水平擴展的基礎

容器讓水平擴展變得自然。你不是在升級一台伺服器的規格，而是在複製容器實例。因為每個容器啟動只需要幾秒鐘，系統可以根據流量快速增減實例。一個起了五個容器的服務，在流量尖峰時可以幾乎即時擴展到五十個容器，流量退潮後再縮回來，只為你真正用到的資源付費。

這和傳統的虛擬機器擴展有根本的差異。VM 啟動要幾分鐘，在流量突增的瞬間，這個速度根本追不上需求。

### 服務隔離

微服務架構下，每個服務跑在自己的容器裡，互相之間的依賴關係清楚劃分。訂單服務用 Python，推薦服務用 Java，通知服務用 Node.js——各自的執行環境完全隔離，互不干擾。一個服務的記憶體洩漏不會影響到其他服務；一個服務要升級某個 library，不需要協調其他服務的測試和部署。

更重要的是，故障隔離。當一個容器崩潰，只有那個服務的那個實例受影響，不是整台伺服器上跑的所有東西。

### 資源效率

傳統的伺服器部署中，你為最高的可能負載配置硬體，然後大部分時間讓那些資源閒著。容器讓多個服務密集地共用同一台機器，每個容器只使用它實際需要的 CPU 和記憶體，空餘的資源可以分給其他容器。

## 容器編排：Kubernetes

管理單台機器上的幾個容器，用 Docker 就夠了。但當你有幾十個服務、幾百個容器實例，分散在幾十台機器上，就需要一個系統來統一管理這些容器的部署、擴展、網路和健康監控。這就是容器編排器（container orchestrator）的工作，而 Kubernetes（簡稱 K8s）是目前最主流的選擇。

> **面試提示**：在系統設計面試中，你通常不需要深入 Kubernetes 的操作細節。但理解它的核心抽象層，能讓你在談到「如何部署和擴展服務」時說得更具體，而不是只說「用容器跑」。

### Kubernetes 的核心概念

**Pod** 是 Kubernetes 的最小部署單位，一個 Pod 裡有一個或多個容器，這些容器共用網路命名空間和儲存掛載。大多數情況下一個 Pod 只跑一個主要容器，有時候會加上 sidecar 容器做輔助工作（例如日誌收集或服務網格的代理）。Pod 是短暫的——它可以在任何時候被停止和重建，所以你不能把任何需要持久化的狀態存在 Pod 裡。

**Deployment** 是你通常用來部署無狀態服務的方式。你告訴 Kubernetes：「我要跑三個 my-service 的 Pod，用 my-service:1.2 這個 image。」Kubernetes 負責確保任何時候都有三個健康的 Pod 在跑。如果一個 Pod 崩潰，它自動啟動新的；如果你要更新版本，它逐步替換舊的 Pod，確保服務不中斷。

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: order-service
```

buildnoat.org

```yaml
spec:
  replicas: 3                      # 保持三個 Pod 同時運行
  selector:
    matchLabels:
      app: order-service
  template:
    metadata:
      labels:
        app: order-service
    spec:
      containers:
      - name: order-service
        image: my-registry/order-service:1.2
        resources:
          requests:
            memory: "256Mi"        # 保證這麼多資源
            cpu: "250m"
          limits:
            memory: "512Mi"        # 最多用這麼多
            cpu: "500m"
        livenessProbe:             # 確認容器還活著
          httpGet:
            path: /health
            port: 3000
          initialDelaySeconds: 10
          periodSeconds: 5
```

**Service** 解決了一個根本問題：Pod 的 IP 地址是不固定的，每次重建都可能改變。Service 提供一個穩定的網路端點，背後自動對應到符合條件的 Pod。客戶端連接到 Service，不需要知道背後有幾個 Pod、它們的 IP 是什麼，Kubernetes 負責把請求分發出去。

**HorizontalPodAutoscaler（HPA）** 根據 CPU 使用率、記憶體壓力或自訂指標，自動調整 Deployment 的 Pod 數量。這讓你的服務能在流量增加時自動擴展，在流量減少時縮回，不需要人工介入。

## 滾動更新與零停機部署

容器化的一個重要優點是部署新版本時可以做到零停機。Kubernetes 的 Deployment 預設採用滾動更新策略：

buildmoat.org

1. 啟動一個跑新版本的 Pod，等它通過 health check
2. 把一個舊版本的 Pod 停掉
3. 重複這個過程，直到所有 Pod 都換成新版本

在整個過程中，始終有 Pod 在提供服務。用戶感覺不到部署的發生。如果新版本有問題，可以立刻回滾到舊版本——因為舊的 image 還在 registry 裡，只是換一行 `image:` 設定的事。

這在容器化之前是非常難做到的事情。傳統的部署方式通常需要停機視窗或複雜的藍綠部署基礎設施。

## 無狀態設計的重要性

容器能夠快速擴展和替換的前提，是服務必須是**無狀態**的（stateless）。

無狀態的服務不在本地儲存任何需要跨請求保留的資料。每個請求帶著它需要的所有資訊，服務處理完就回傳結果，不留下任何痕跡。這樣的服務可以任意地新增、移除、替換實例，負載平衡器把請求打到任何一個實例，結果都相同。

有狀態的設計會把容器的好處破壞殆盡。假設你把 session 資料存在容器的本地記憶體裡：用戶第一個請求打到容器 A，session 建立在那裡；第二個請求被負載平衡器打到容器 B，容器 B 找不到這個 session，用戶被登出了。更糟的是，容器 A 重啟後，所有存在它記憶體裡的 session 全部消失。

正確的做法是把狀態外部化：

- **Session / 用戶狀態** → Redis
- **持久化資料** → PostgreSQL、MySQL 或其他資料庫
- **檔案、媒體** → S3 或其他物件儲存
- **服務間的設定** → 環境變數或 ConfigMap

容器本身是短暫的（ephemeral），它隨時可以被停止和重建，不帶走任何東西。所有需要保留的東西都存在容器外面。

> **面試提示**：當你在設計中提到容器化時，面試官很可能問「那你怎麼處理狀態？」。主動說明你的服務是無狀態設計，狀態存在 Redis 或資料庫裡，展示你理解這個基本原則。

## 容器的網路模型

在容器環境中，服務之間的溝通方式和傳統部署有所不同，理解這一點能讓你更清楚地設計服務間的通訊架構。

buildmeat.org

**同一台機器上的容器**可以透過 Docker 的虛擬網路直接溝通，不需要出去繞一圈公共網路。這讓同一個 Pod 裡的容器（例如主服務和 sidecar）可以用 `localhost` 互相呼叫。

**跨機器的服務間通訊**透過 Kubernetes 的 Service 物件進行。每個 Service 有一個 DNS 名稱，例如 `order-service.production.svc.cluster.local`，在 cluster 內的任何服務都可以用這個名稱找到它，而不需要知道任何 Pod 的實際 IP 地址。

**對外暴露服務**通常透過 Ingress Controller 來做。Ingress 是 Kubernetes 裡管理外部 HTTP/HTTPS 流量的元件，它根據路徑或 host 名稱把流量路由到對應的 Service，也可以在這一層做 TLS 終止和流量限速。

# 什麼時候在面試裡用這些

## 水平擴展的討論

每當面試官問「這個服務怎麼擴展？」，容器就是你的工具。

「我們把這個 API service 容器化，部署在 Kubernetes 上。設定 HorizontalPodAutoscaler，當 CPU 使用率超過 70% 時自動增加 Pod 數量。由於服務是無狀態的，負載平衡器可以把請求打到任何一個 Pod，擴展和縮小都是幾秒鐘的事。」

這比「加更多伺服器」具體得多，也展示了你對現代部署架構的理解。

## 微服務架構的落地

提到微服務時，容器是讓架構真正落地的手段。

「每個微服務有自己的 Docker image，獨立部署在 Kubernetes 上。服務之間只透過 API 溝通，不共用資料庫。這讓每個服務可以用最適合它的語言和技術棧，也讓一個服務的故障不會連鎖影響到其他服務。」

## 部署可靠性的討論

「我們用滾動更新策略部署新版本——先讓新版本的 Pod 通過 health check，再逐步替換舊版本。整個過程中始終有 Pod 在服務，用戶感覺不到部署。如果新版本有問題，一個指令就能回滾到上一個版本。」

## 常見面試情境

**電商系統（高流量促銷活動）**：「雙十一期間，商品瀏覽和結帳服務的流量可能暴增十倍。因為這些服務是無狀態的容器，Kubernetes HPA 可以在幾秒內把 Pod 數量從 10 個擴展到 100 個，活動結束後自動縮回，只付尖峰期的資源費用。」

buildmoat.org

**影片串流平台**：「轉碼服務是計算密集型的，和面向用戶的 API 有完全不同的擴展需求。我會把它們分開容器化——API 服務用通用型機器快速擴展，轉碼容器跑在 GPU 優化的機器上，按需要的數量啟動，任務完成後停掉，不浪費 GPU 資源。」

**多租戶 SaaS**：「我們用 Kubernetes namespace 把不同客戶的服務隔離開來。每個 namespace 有自己的資源配額，確保一個客戶的異常流量不會影響其他客戶的體驗。」

## 常見的 Deep Dive 問題

### 「容器崩潰了怎麼辦？」

這是面試官測試你對容器化真實理解的常見問題。

在 Kubernetes 裡，Deployment 負責保證指定數量的 Pod 始終在運行。如果一個 Pod 崩潰，Kubernetes 的 controller loop 會在幾秒內偵測到（透過 liveness probe），並自動啟動一個新的 Pod 替換它。在大多數情況下，這比人工反應快得多，而且是全自動的。

Liveness probe 告訴 Kubernetes「這個容器還活著嗎？」；Readiness probe 則告訴它「這個容器準備好接收流量了嗎？」兩者的差別很重要——一個服務可能活著但還在暖機，此時不應該讓它接流量；或者一個服務雖然沒崩潰，但連接不到資料庫，那請求打過去也是白打。

```
livenessProbe:
  httpGet:
    path: /health/live      # 只要行程還活著就回 200
    port: 3000
  failureThreshold: 3       # 連續失敗三次才重啟

readinessProbe:
  httpGet:
    path: /health/ready     # 資料庫連線正常才回 200
    port: 3000
  failureThreshold: 1       # 一次失敗就從 Service 的後端移除
```

### 「怎麼處理容器的設定和機密？」

容器 image 本身不應該包含任何環境相關的設定，更不應該包含密碼、API key 這類機密。一個 image 要能在開發、測試、生產三個環境使用，差別只在設定，而不是建置三份不同的 image。

buildmeat.org

**設定**透過環境變數或 Kubernetes ConfigMap 注入：

```
env:
  - name: DATABASE_HOST
    valueFrom:
      configMapKeyRef:
        name: order-service-config
        key: db_host
  - name: CACHE_TTL
    value: "300"
```

**機密**（密碼、API key、憑證）透過 Kubernetes Secret 或外部的機密管理服務（例如 AWS Secrets Manager、HashiCorp Vault）注入。機密不應該存在 image 裡，也不應該出現在 git 記錄中：

```
env:
  - name: DATABASE_PASSWORD
    valueFrom:
      secretKeyRef:
        name: order-service-secrets
        key: db_password
```

buildmoat.org

## 「容器和 VM 在你的設計裡什麼時候各自適合？」

這個問題考的是你的判斷力，而不是技術細節。

**用容器的場景**：大多數的微服務、API、worker、排程任務。啟動快、密度高、CI/CD 流程順暢。90% 的工作負載都適合容器。

**用 VM 的場景**：

- **強安全隔離**：多租戶環境下，讓不同客戶的程式碼跑在完全隔離的 VM 裡，kernel 層面的隔離是容器給不了的。AWS Lambda 的底層用 Firecracker microVM，Fargate 的隔離也是 VM 級別的。
- **跑不同 OS**：某個服務必須跑在 Windows 上，但你的 cluster 是 Linux——就用 VM。
- **Legacy 應用**：有些老系統無法容器化（例如依賴特定的 kernel 版本或低層次硬體操作），VM 是唯一現實的選項。

面試中清楚說出這個判斷，比說「我們全部用容器」更有說服力。

### 「服務更新時怎麼保證零停機？」

除了前面提到的 Kubernetes 滾動更新，還有幾個需要注意的細節：

**資料庫遷移（migration） 要向後兼容**：更新服務之前先做 migration，確保新舊版本的程式碼都能在新的資料庫 schema 上正確運作。不能同時做「加了一個 not null 欄位」的 schema 變更和服務更新——滾動更新期間，舊版本的容器會因為這個欄位崩潰。

**版本化的 API**：如果一個 API 更新有破壞性變更，保留舊版本（`/v1/...`）一段時間，等所有客戶端都切換到新版本後再下掉，而不是直接改掉讓所有人同時崩潰。

**Graceful shutdown**：收到停止訊號（SIGTERM）時，容器不應該立刻退出，而是先拒絕新的請求、等待正在處理的請求完成，然後再乾淨地退出。這避免了滾動更新過程中請求在半途被中斷的問題。

## 總結

容器的核心價值在三件事：**一致性**（在任何環境都一樣跑）、**隔離性**（服務之間互不干擾）、**效率**（快速啟動、高密度部署）。

在系統設計面試中，提到容器的時機通常是在討論水平擴展或微服務部署時。關鍵是說清楚你設計的服務為什麼適合容器化——通常是因為它是無狀態的，可以任意複製實例——然後說明 Kubernetes 如何管理這些實例的生命週期和擴展。

不要把容器當成萬靈丹。有狀態的服務需要額外的設計（PersistentVolume、StatefulSet），強安全隔離的場景可能需要 VM。展示你知道容器適合什麼、不適合什麼，才是真正成熟的架構判斷。

容器讓「每秒多開幾個實例」從難事變成常規操作。在一個需要應對流量突增的系統裡，這個能力是設計的地基，而不是優化的後話。

buildmeat.org